

Documentație proiect

Snake cu Arduino Mega 2560

Mihuț Cristian-Mihai
Grupa 30236

1 Scopul codului

Codul implementat în cadrul proiectului Snake IoT are rolul de a realiza un joc Snake distribuit pe mai multe dispozitive. Logica jocului, afișarea și funcțiile IoT sunt separate, iar codul permite comunicarea între aceste componente folosind mesaje seriale.

2 Structura generală a codului Python

Codul Python rulează pe PC și este responsabil de:

- afișarea grafică a jocului;
- citirea datelor de la Arduino Mega;
- trimiterea scorului și a setărilor către ESP32;
- gestionarea interfeței (scor, pauză, game over).

Codul este organizat în următoarele părți principale:

1. configurare și inițializare;
2. funcții grafice;
3. ecran de încărcare;
4. bucla principală de joc.

3 Inițializare și configurare

La începutul codului sunt definite constantele:

- dimensiunea grilei;
- dimensiunea blocurilor;
- culorile utilizate;
- porturile seriale și viteza de comunicare.

De asemenea, este inițializată biblioteca `pygame`, fereastra de joc și fonturile utilizate pentru afișare.

4 Conectarea la Arduino Mega și ESP32

Funcția `init_connections()` încearcă să deschidă 2 conexiuni seriale:

- una cu Arduino Mega, care este obligatorie;
- una cu ESP32, care este opțională.

Dacă Arduino Mega nu este detectat, aplicația se închide. Dacă ESP32 nu este conectat, jocul poate rula în continuare, dar fără funcțiile IoT (configurare live și salvare scor maxim).

5 Funcțiile de desenare

Codul conține mai multe funcții dedicate desenării:

- desenarea grilei;
- desenarea mărului;
- desenarea segmentelor șarpelui;
- desenarea umbrelor și efectelor vizuale.

Aceste funcții nu modifică logica jocului, ci doar afișează datele primite de la Arduino Mega într-un mod vizual.

6 Ecranul de încărcare

La pornire, aplicația afișează un ecran de încărcare. În această fază:

- se solicită scorul maxim de la ESP32 folosind comanda `GET_HIGHSORE`;
- se afișează un mesaj de conectare la rețea;
- se așteaptă răspunsul de tip `HIGHSORE`.

După primirea scorului sau după un timeout, jocul pornește.

7 Bucla principală de joc

Bucla principală rulează continuu până la închiderea aplicației.

7.1 Citirea datelor de la Arduino Mega

Arduino Mega transmite periodic către PC starea completă a jocului sub forma unei linii de text:

```
DATA:x_food,y_food;dir;snake_body;score;game_over;speed;pause
```

Semnificația câmpurilor este următoarea:

- `x_food,y_food` : poziția mărului pe grila de joc;
- `dir` : direcția de deplasare a șarpelui (utilizată intern);
- `snake_body` : lista coordonatelor segmentelor șarpelui:

`x1,y1,x2,y2,...`

primul element fiind capul;

- `score` : scorul curent al jocului;
- `game_over` : indicator logic pentru final de joc;
- `speed` : viteza curentă a jocului (întârziere între pași);
- `pause` : indicator logic pentru pauză.

Codul Python parsează aceste date și le utilizează exclusiv pentru afișare.

7.2 Actualizarea interfeței

După interpretarea datelor:

- ecranul este curățat;
- se desenează grila, mărul și șarpele;
- se afișează scorul curent și scorul maxim;
- se afișează mesaje speciale pentru pauză sau game over.

7.3 Comunicarea cu ESP32

ESP32 este responsabil pentru funcțiile IoT și comunică bidirecțional cu aplicația Python.

Mesaje trimise către ESP32

- `GET_HIGHSORE` : solicită scorul maxim salvat;
- `LIVE:score` : trimite scorul curent în timp real;
- `LIVE_SPEED:speed` : transmite viteza jocului;
- `SET_SCORE:value` : actualizează scorul maxim.

Mesaje primite de la ESP32

- `HIGHSCORE:value` : scorul maxim salvat în baza de date;
- `CONFIG:SPD=x;GRID=w,h;COLOR=name;WALLS=0/1` – setări de joc:
 - `SPD` : viteza inițială a jocului;
 - `GRID` : dimensiunea grilei;
 - `COLOR` : culoarea șarpelui;
 - `WALLS` : activarea coliziunilor cu pereții.
- mesaje de tip informativ (ex: adresă IP).

Setările primite sunt transmise mai departe către Arduino Mega, care aplică modificările în logica jocului.

8 Principiul de funcționare al întregului sistem

Funcționarea proiectului poate fi rezumată astfel:

1. Arduino Mega calculează logica jocului;
2. PC-ul afișează starea primită;
3. ESP32 gestionează scorurile și configurările;
4. toate componentele comunică prin mesaje seriale.

Această abordare permite separarea clară între logică, afișare și funcții IoT.

9 Concluzie

Codul proiectului Snake IoT este construit modular și ușor de urmărit. Separarea responsabilităților între Arduino Mega, PC și ESP32 face sistemul stabil, extensibil și ușor de depanat. Proiectul demonstrează integrarea eficientă a jocurilor cu sisteme embedded și tehnologii IoT.